République du Bénin

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Nationale des Sciences, Technologies,
Ingénierie et Mathématiques d'Abomey (UNSTIM)

École Nationale des Génies Mathématiques et Modélisation (ENSGMM)

Implémentation du modèle d'infection par le VIH-SIDA

Modèles intra-hôte et de population : analyse mathématique, stabilité et résolution numérique avec BLAS, LAPACK et Intel MKL en Python

Équipe de projet – Groupe 5

ADOGOUN Déo Gracias Mahugnon

BIAHOU Ahossou J. Marie-Clémenceau

SYLLA IMOROU Izou Dine

Filière : ENSGMM1 • **UE :** Outils de calcul scientifique

Année académique : 2026–2027

Sous la direction de : Dr. Carlos AGOSSOU

Table des matières

Introduction **1**

1 Présentation d'Intel MKL (oneMKL) **2**
1.1 Qu'est-ce qu'Intel MKL ? . . . 2
1.2 Domaines couverts . . . 2
1.3 Avantages et alternatives du marché . . . 2

2 BLAS (Basic Linear Algebra Subprograms) **4**
2.1 Origine et philosophie de conception . . . 4
2.2 Les trois niveaux de BLAS et l'intensité arithmétique . . . 4
2.3 Conventions de nommage universelles . . . 4

3 LAPACK (Linear Algebra PACKage) **5**
3.1 Présentation et Algorithmes par Blocs . . . 5
3.2 Fonctionnalités et routines pilotes . . . 5
3.3 Schéma vectoriel de la pile logicielle . . . 5

4 Installation, Configuration et Utilisation **7**
4.1 Configuration sous Python et vérification du backend . . . 7
4.2 Appel direct de LAPACK depuis Python . . . 7

5 Le VIH-SIDA : Contexte Biologique et Échelles **8**
5.1 Physiopathologie de l'infection . . . 8
5.2 Nécessité de deux échelles de modélisation . . . 8

6 Le Modèle Intra-Hôte : Formulation et Analyse **9**
6.1 Formulation mathématique . . . 9
6.2 Positivité, Bornitude et Nombre de Reproduction $\mathcal{R}_0$ . . . 9
6.3 Stabilité Locale et Globale . . . 10

7 Méthodes Numériques et LAPACK **11**
7.1 Systèmes linéaires, erreur directe et conditionnement . . . 11
7.2 Équilibrage de la matrice Jacobienne avec DGEEQU . . . 11
7.3 Méthode de Newton avec DGESV et calcul spectral DGEEV . . . 11

8 Intégration Numérique des Équations **13**
8.1 Raideur et instabilité des schémas explicites . . . 13

9 Résultats et Interprétation Médicale **14**
9.1 Primo-infection et portrait de phase . . . 14
9.2 Efficacité thérapeutique et rebond viral . . . 14

10 Modèle Épidémiologique de Population : S, I_1, I_2 **16**
10.1 Formulation conforme à la diapositive du cours 16
10.2 Nombre de reproduction de base et équilibres 16
10.3 Scénarios d'intervention de santé publique 17

11 Benchmark Haute Performance BLAS / LAPACK **18**
11.1 Protocole de mesure 18

Conclusion et Perspectives **19**

Références Bibliographiques **20**

A Code Source du Mini-Projet (vih_lapack.py) **21**

B Code Source du Benchmark (benchmark_blas.py) **24**

C Guide d'Exécution et Reproductibilité **26**

Introduction

Résumé Exécutif & Comprendre le Projet en 1 Minute

Ce rapport présente une étude complète de la dynamique de l'infection par le virus de l'immunodéficience humaine (VIH), articulée à deux échelles fondamentales : l'échelle du patient (**modèle intra-hôte** T, I, V) et l'échelle d'une communauté (**modèle épidémiologique de population** S, I_1, I_2).

L'enjeu informatique et scientifique : Les équations différentielles sous-jacentes sont non-linéaires et fortement raides (*stiff*). Les résoudre sur des horizons de temps longs sans instabilité exige des schémas implicites et des résolutions matricielles directes. Nous montrons comment la pile logicielle formée par **BLAS**, **LAPACK** (routines `DGESV`, `DGEEV`, `DGEEQU`, `DGESDD`) et **Intel MKL** permet d'atteindre des accélérations de calcul spectaculaires (facteur $\times 200$ à $\times 900$) face aux implémentations naïves.

Contexte. Le calcul scientifique moderne repose sur quelques briques logicielles d'algèbre linéaire dense, **BLAS** et **LAPACK**, dont des implémentations très optimisées existent (**Intel MKL**, OpenBLAS, BLIS). Presque tous les environnements de calcul scientifique (NumPy, SciPy, MATLAB, R, Julia) les utilisent en arrière-plan.

Objectifs de ce rapport.

1. Présenter l'architecture des bibliothèques BLAS, LAPACK et Intel MKL ;
2. Montrer comment les configurer et les appeler directement depuis Python ;
3. Les mettre en pratique sur un cas d'étude médical à deux échelles : celle du patient (modèle intra-hôte de Nowak et May / Perelson) et celle de la population (modèle épidémiologique à deux stades avec mortalités) ;
4. Mesurer leurs performances numériques et leur scalabilité multi-threads face à des implémentations ordinaires.

Organisation. Les chapitres 1 à 4 détaillent les bibliothèques logicielles. Le chapitre 5 expose le contexte biologique. Les chapitres 6 à 9 traitent le modèle intra-hôte (T, I, V). Le chapitre 10 étudie le modèle de population (S, I_1, I_2). Le chapitre 11 présente les résultats des benchmarks d'accélération. Les codes sources complets sont reproduits en annexes.

Chapitre 1

Présentation d'Intel MKL (oneMKL)

1.1 Qu'est-ce qu'Intel MKL ?

Intel oneAPI Math Kernel Library (**oneMKL**, anciennement Intel MKL) est une bibliothèque industrielle de routines mathématiques ultra-optimisées pour le calcul scientifique et l'ingénierie sur architectures x86-64. Elle tire le meilleur parti des jeux d'instructions vectorielles des microprocesseurs (SSE, AVX2, AVX-512, Intel AMX) et du parallélisme multi-cœurs via OpenMP et Intel TBB. Au démarrage, son *CPU Dispatcher* détecte dynamiquement le modèle exact du processeur hôte pour router l'exécution vers les micro-noyaux assembleurs les plus rapides.

1.2 Domaines couverts

— **Algèbre linéaire dense :** BLAS (niveaux 1, 2, 3), LAPACK et ScaLAPACK (calcul parallèle distribué sur grappes MPI).
— **Algèbre linéaire creuse :** Sparse BLAS et solveur direct multithread PARDISO.
— **Transformées de Fourier rapides (FFT) :** Interface compatible FFTW multidimensionnelle.
— **Mathématiques vectorielles (VML) :** Fonctions transcendantes (`vdExp`, `vdLn`, `vdSin`, `vdInvSqrt`) calculées sur des vecteurs entiers en un cycle SIMD.
— **Statistiques et nombres aléatoires (VSL) :** Générateurs pseudo-aléatoires haute performance.

1.3 Avantages et alternatives du marché

Intel MKL offre les meilleures performances crêtes sur processeurs Intel et compatibles AMD. Le tableau 1.1 synthétise le paysage des bibliothèques concurrentes.

TABLE 1.1 – Principales alternatives à Intel MKL.

Bibliothèque	**Nature / Licence**	**Spécificités techniques**
OpenBLAS	Libre (BSD)	Dérivée de GotoBLAS2 ; livrée par défaut avec NumPy/SciPy sur Linux/macOS.
BLIS	Libre (BSD)	Cadre modulaire développé à l'UT Austin avec micro-noyaux portables.
ATLAS	Libre	Optimisation automatique hors-ligne lors de l'étape de compilation.
Netlib	Libre (Référence)	Code Fortran de référence étalon, sans vectorisation matérielle.
Intel oneMKL	Propriétaire gratuit	Code assembleur spécifique, vectorisation AVX-512 et threading OpenMP.

Chapitre 2

BLAS (Basic Linear Algebra Subprograms)

2.1 Origine et philosophie de conception

Créée à la fin des années 1970 par Lawson et al., BLAS est une spécification d'opérations élémentaires. Sa standardisation garantit la portabilité : l'application appelle la même routine (`DGEMM`), et le constructeur fournit le binaire optimisé pour son matériel.

2.2 Les trois niveaux de BLAS et l'intensité arithmétique

Le tableau 2.1 récapitule la taxonomie des trois niveaux.

TABLE 2.1 – Les trois niveaux de BLAS et leur intensité arithmétique.

Niveau	Opérations	Données	Calculs	Intensité	Exemples phares
1	Vecteur-Vecteur	$O(n)$	$O(n)$	$O(1)$ (Faible)	`DAXPY` ($y \leftarrow \alpha x + y$), `DDOT`, `DNRM2`
2	Matrice-Vecteur	$O(n^2)$	$O(n^2)$	$O(1)$ (Faible)	`DGEMV` ($y \leftarrow \alpha Ax + \beta y$), `DGER`, `DTRSV`
3	Matrice-Matrice	$O(n^2)$	$O(n^3)$	$\mathbf{O(n)}$ (Élevée)	`DGEMM` ($C \leftarrow \alpha AB + \beta C$), `DSYRK`, `DTRSM`

Le secret de BLAS 3 : le cache tiling. Dans les niveaux 1 et 2, pour chaque nombre lu en mémoire, le CPU n'exécute qu'une seule opération : le bus mémoire sature immédiatement (*Memory-Bound*). Dans le niveau 3, pour n^2 nombres chargés, on réalise n^3 opérations. En découpant les matrices en sous-blocs de taille $B \times B$ calibrés pour tenir dans le cache L1, chaque élément est réutilisé B fois sur place, atteignant la vitesse de crête du processeur (*Compute-Bound*).

2.3 Conventions de nommage universelles

La signature suit la structure `[Type][Matrice][Opération]` :
— **Type :** `S` (réel 32 bits), `D` (réel 64 bits double), `C` (complexe simple), `Z` (complexe double).
— **Structure :** `GE` (générale), `SY` (symétrique), `TR` (triangulaire), `GB` (bande).
— **Opération :** `MM` (produit matrice-matrice), `MV` (matrice-vecteur), `RK` (mise à jour de rang k).

Chapitre 3

LAPACK (Linear Algebra PACKage)

3.1 Présentation et Algorithmes par Blocs

Apparue au début des années 1990 pour succéder à LINPACK et EISPACK, LAPACK réorganise les algorithmes d'algèbre linéaire avancée en **algorithmes par blocs**. Au lieu de factoriser colonne par colonne, LAPACK factorise un panneau vertical puis met à jour tout le reste de la matrice d'un coup avec `DGEMM` (BLAS 3), garantissant une efficacité maximale.

3.2 Fonctionnalités et routines pilotes

Table 3.1 – Principales fonctionnalités de LAPACK.

Problème mathématique	**Routine Pilote**	**Algorithme sous-jacent**
Système général $Ax = b$	`DGESV`	Factorisation LU avec pivotage partiel ($PA = LU$)
Système symétrique défini positif	`DPOSV`	Factorisation de Cholesky ($A = LL^T$)
Moindres carrés $\min \|Ax - b\|_2$	`DGELS`	Décomposition orthogonale QR
Valeurs propres symétriques	`DSYEV`	Tridiagonalisation + Divide & Conquer
Valeurs propres générales	`DGEEV`	Réduction de Hessenberg + QR décalé
Décomposition SVD ($A = U\Sigma V^T$)	`DGESVD`	Bidiagonalisation QR
Équilibrage de matrice	`DGEEQU`	Équilibrage de lignes et colonnes
Estimation conditionnement	`DGECON`	Estimation de $\|A^{-1}\|_1$ sans inversion

3.3 Schéma vectoriel de la pile logicielle

La figure 3.1 illustre l'empilement logiciel menant de l'application scientifique jusqu'au matériel processeur.

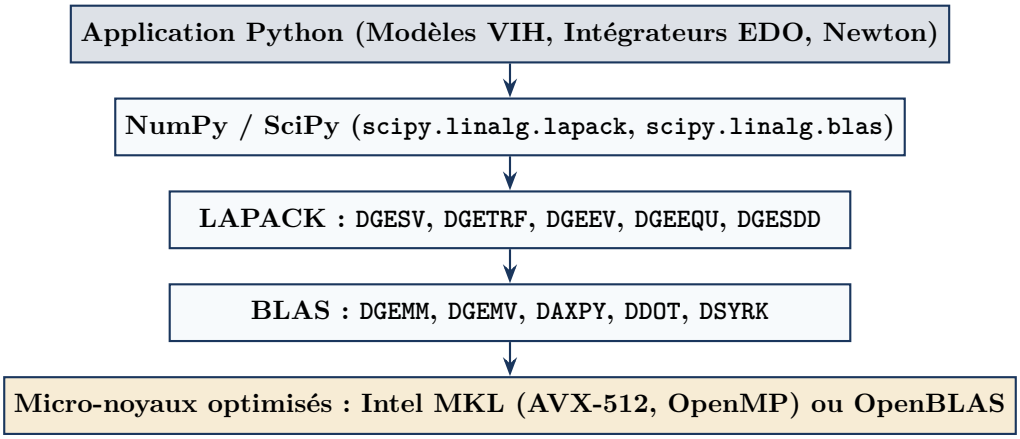


FIGURE 3.1 – Pile logicielle : de l'application scientifique aux noyaux assembleurs optimisés.

Chapitre 4

Installation, Configuration et Utilisation

4.1 Configuration sous Python et vérification du backend

NumPy et SciPy peuvent être liés soit à OpenBLAS (via les roues standard `pip`), soit à Intel MKL (via la distribution `conda`). La vérification dynamique de la bibliothèque active s'effectue simplement :

```python
import numpy as np
from threadpoolctl import threadpool_info

print(np.show_config()) # Configuration BLAS/LAPACK de NumPy
print(threadpool_info()) # Bibliothèque liée, version et nombre de threads
```

4.2 Appel direct de LAPACK depuis Python

SciPy offre une interface de bas niveau zéro-copie vers les routines Fortran de LAPACK :

```python
from scipy.linalg import lapack
import numpy as np

A = np.asfortranarray([[4.0, 1.0], [2.0, 3.0]])
b = np.array([1.0, 2.0])

# Résolution LU directe Ax = b
lu, piv, x, info = lapack.dgesv(A, b)
assert info == 0, "Échec de factorisation"

# Calcul des valeurs propres (parties réelles et imaginaires)
wr, wi, vl, vr, info = lapack.dgeev(A)
valeurs_propres = wr + 1j * wi
```

Chapitre 5

Le VIH-SIDA : Contexte Biologique et Échelles

5.1 Physiopathologie de l'infection

Le Virus de l'Immunodéficience Humaine (VIH) cible spécifiquement les lymphocytes auxiliaires T CD4+, pivot de la coordination immunitaire. Un virion libre pénètre dans une cellule T saine, rétrotranscrit son ARN en ADN proviral et détourne la cellule pour produire de nouveaux virions avant sa mort par apoptose ou cytotoxicité. Chez un patient non traité, l'infection évolue selon trois phases :

1. **Phase aiguë (semaines) :** Pic de charge virale et effondrement temporaire des CD4+ ;
2. **Phase chronique (années) :** Équilibre dynamique stationnaire (*set point*) ;
3. **Stade SIDA :** Chute irréversible des CD4+ en dessous du seuil critique de 200 cellules/μL, ouvrant la voie aux infections opportunistes mortelles.

5.2 Nécessité de deux échelles de modélisation

— **Échelle intra-hôte (dans le patient) :** Modèle (T, I, V) en jours. Analyse de la charge virale, de la résilience et de l'efficacité thérapeutique η.
— **Échelle de population (épidémiologique) :** Modèle (S, I_1, I_2) en années. Analyse de la prévalence et des interventions de santé publique.

Chapitre 6

Le Modèle Intra-Hôte : Formulation et Analyse

6.1 Formulation mathématique

Le modèle de Nowak-May-Perelson avec traitement antirétroviral d'efficacité $\eta \in [0,1]$ s'écrit :

$$\begin{cases} \dfrac{dT}{dt} = s - dT - (1-\eta)\beta TV \\ \dfrac{dI}{dt} = (1-\eta)\beta TV - \delta I \\ \dfrac{dV}{dt} = pI - cV \end{cases} \tag{6.1}$$

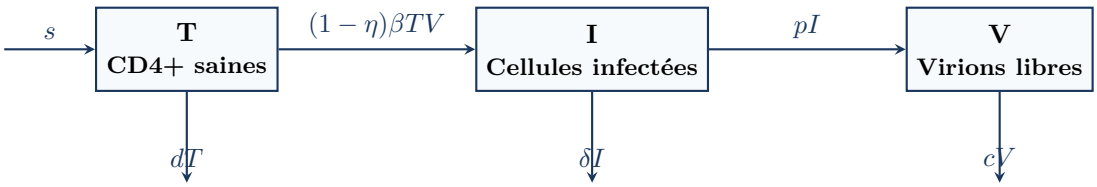


FIGURE 6.1 – Schéma de transfert compartimental du modèle intra-hôte.

TABLE 6.1 – Paramètres biomédicaux du modèle intra-hôte.

Symbole	Signification	Valeur	Unité	Source de calibration
s	Production de CD4+ saines	10	cellules/μL/jour	Hypothèse physiologique
d	Mortalité naturelle des CD4+	0,01	jour^{-1}	Perelson & Nelson ($T_0 = s/$
β	Infectivité virale	$4{,}6 \times 10^{-5}$	μL/virion/jour	Calibré pour $\mathcal{R}_0 = 4$
δ	Mortalité des cellules infectées	0,50	jour^{-1}	Demi-vie virale $\approx$ 1,4 jour
p	Production de virions par cellule	1000	virions/cellule/jour	Nowak & May
c	Clairance des virions libres	23	jour^{-1}	Demi-vie plasmatique $\approx$ 5,5
η	Efficacité thérapeutique ARV	0 à 1	–	Paramètre de contrôle clinic

6.2 Positivité, Bornitude et Nombre de Reproduction $\mathcal{R}_0$

Proposition 6.1. *Pour toute condition initiale positive, les solutions de (6.1) existent pour tout $t \geq 0$, restent strictement positives, et vérifient* $\limsup_{t\to\infty} T(t) \leq s/d$.

Calcul du $\mathcal{R}_0$ par la Matrice de Prochaine Génération (NGM) : Posons $b = (1-\eta)\beta$. Pour le sous-système infecté $(I, V)^T$, la matrice de transmission F et la matrice de transition V_m s'écrivent :

$$F = \begin{pmatrix} 0 & bT_0 \\ 0 & 0 \end{pmatrix}, \quad V_m = \begin{pmatrix} \delta & 0 \\ -p & c \end{pmatrix} \implies V_m^{-1} = \begin{pmatrix} 1/\delta & 0 \\ p/(\delta c) & 1/c \end{pmatrix}$$

Le rayon spectral de FV_m^{-1} donne le nombre de reproduction de base :

$$\mathcal{R}_0(\eta) = \frac{(1-\eta)\beta p T_0}{\delta c} = (1-\eta)\mathcal{R}_0(0), \quad \text{avec } \eta_c = 1 - \frac{1}{\mathcal{R}_0(0)} = 75{,}0\,\% \tag{6.2}$$

6.3 Stabilité Locale et Globale

Théorème 6.1 (Stabilité du DFE)**.** *L'équilibre sans infection $E_0 = (T_0, 0, 0)$ est globalement asymptotiquement stable (GAS) si $\mathcal{R}_0 \leq 1$, et instable si $\mathcal{R}_0 > 1$.*

Démonstration. Considérons la fonction de Lyapunov $L = I + \frac{\delta}{p}V \geq 0$. Sa dérivée le long des trajectoires vaut $\dot{L} = (bT - \frac{\delta c}{p})V$. Comme $T \leq T_0$, on a $\dot{L} \leq \frac{\delta c}{p}(\mathcal{R}_0 - 1)V \leq 0$ lorsque $\mathcal{R}_0 \leq 1$. Le principe d'invariance de LaSalle conclut à la stabilité globale du DFE. □

Théorème 6.2 (Stabilité de l'Équilibre Endémique EE)**.** *Pour $\mathcal{R}_0 > 1$, l'équilibre endémique $E^* = (T^*, I^*, V^*)$ existe, est unique, et est globalement asymptotiquement stable dans le domaine positif avec :*

$$T^* = \frac{T_0}{\mathcal{R}_0}, \quad I^* = \frac{s}{\delta}\left(1 - \frac{1}{\mathcal{R}_0}\right), \quad V^* = \frac{p}{\delta}I^* = \frac{ps}{c\delta}\left(1 - \frac{1}{\mathcal{R}_0}\right)$$

Chapitre 7

Méthodes Numériques et LAPACK

7.1 Systèmes linéaires, erreur directe et conditionnement

La résolution de $Ax = b$ via `DGETRF` et `DGETRS` (regroupés dans `DGESV`) réalise une élimination de Gauss rétrograde-stable. La figure 7.1 illustre l'évaluation du conditionnement sur des matrices de Hilbert : le résidu reste au niveau de la précision machine ($\approx 10^{-16}$), tandis que l'erreur directe croît proportionnellement à $\kappa_1(A)\,\varepsilon_{mach}$.

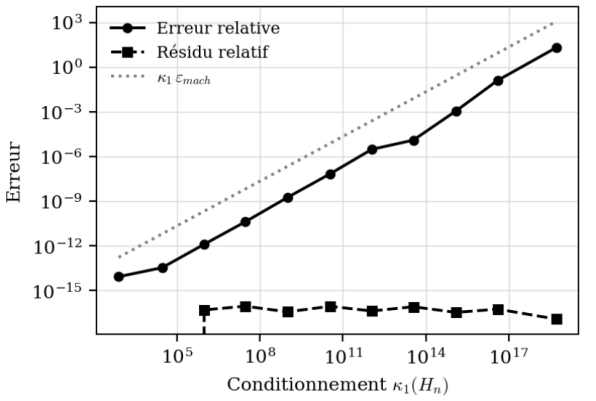


FIGURE 7.1 – Erreur relative et résidu vs conditionnement.

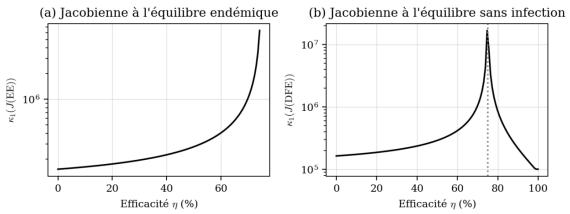


FIGURE 7.2 – Conditionnement de la Jacobienne $\kappa_1(J)$.

7.2 Équilibrage de la matrice Jacobienne avec DGEEQU

À l'équilibre endémique, la Jacobienne présente un conditionnement élevé $\kappa_1(J(EE)) \approx 1{,}52 \times 10^5$ (figure 7.2). Cela résulte de l'écart d'échelle entre cellules ($s = 10$) et virions ($p = 1000$). L'appel à la routine LAPACK `DGEEQU` calcule les facteurs d'échelle optimaux de lignes (R) et colonnes (C) :

$$\widehat{J} = R \cdot J \cdot C$$

Cet équilibrage ramène le conditionnement de $1{,}5 \times 10^5$ à $\kappa_1(\widehat{J}) \approx 2{,}4 \times 10^2$, éliminant tout risque d'instabilité numérique.

7.3 Méthode de Newton avec DGESV et calcul spectral DGEEV

La recherche de l'équilibre $f(y) = 0$ par la méthode de Newton calcule à chaque pas :

$$J(y_k)\delta_k = -f(y_k), \quad y_{k+1} = y_k + \delta_k$$

Le système linéaire 3×3 est résolu par `DGESV`. La convergence est quadratique en 5 à 6 itérations (figure 7.3).

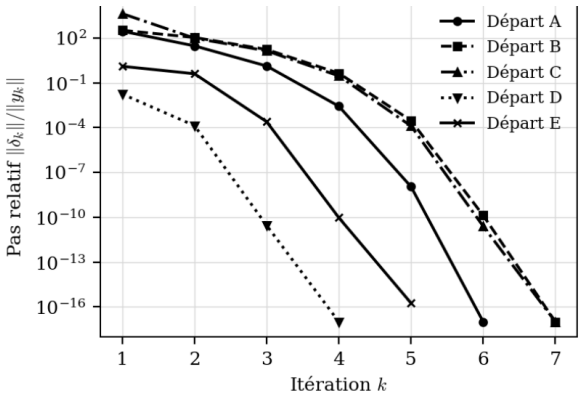


FIGURE 7.3 – Convergence quadratique de la méthode de Newton avec `DGESV`.

La validation des valeurs propres obtenues par `DGEEV` face aux racines analytiques du polynôme caractéristique montre un écart maximal de $1{,}4\times10^{-14}$ (précision machine). À l'EE sans traitement, les valeurs propres sont :

$\lambda_1 = -23{,}5006$ (mode rapide), $\lambda_{2,3} = -0{,}0197 \pm 0{,}1196\, i$ (oscillations amorties de période 53 jours)

Chapitre 8

Intégration Numérique des Équations

8.1 Raideur et instabilité des schémas explicites

Le système est fortement raide : le rapport des échelles de temps $\max|\mathrm{Re}\,\lambda| / \min|\mathrm{Re}\,\lambda| \approx 1{,}2 \times 10^3$. Pour Euler explicite, la stabilité exige $h < 2/|\lambda_{\max}| \approx 0{,}080$ jour. Au-delà ($h = 0{,}1$ j), le schéma explose (figure 8.1a). En revanche, Euler implicite résolu à chaque pas par `DGESV` reste inconditionnellement stable (figure 8.1b).

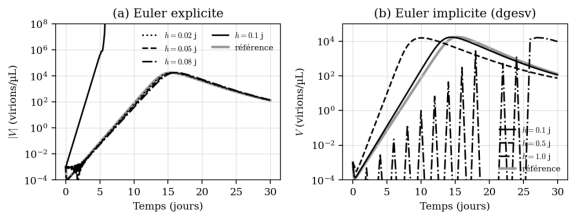


FIGURE 8.1 – Stabilité comparée : Euler explicite vs implicite.

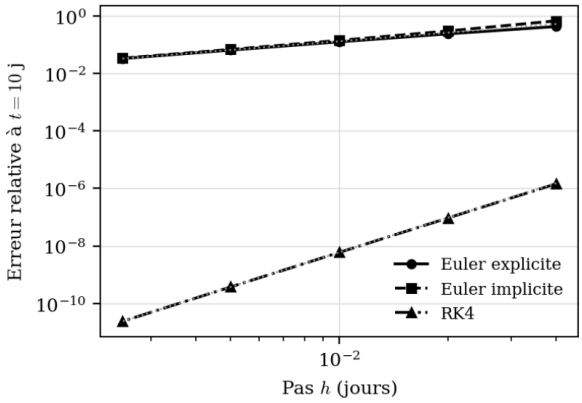


FIGURE 8.2 – Ordres de convergence numérique observés.

Chapitre 9

Résultats et Interprétation Médicale

9.1 Primo-infection et portrait de phase

La figure 9.1 montre la dynamique de primo-infection : pic viral aigu à 15,4 jours (17 373 virions/μL), chute des CD4+ à 49 cellules/μL, puis stabilisation oscillatoire vers le point de consigne $V^* = 652{,}2$. La convergence en spirale vers l'EE est attestée par le portrait de phase (figure 9.2).

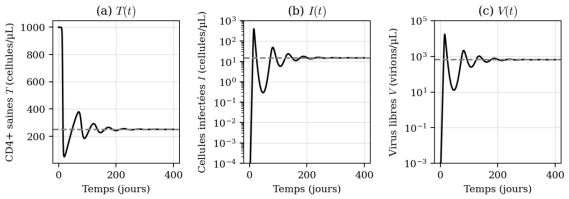


FIGURE 9.1 – Infection primaire : CD4+, cellules infectées et virus.

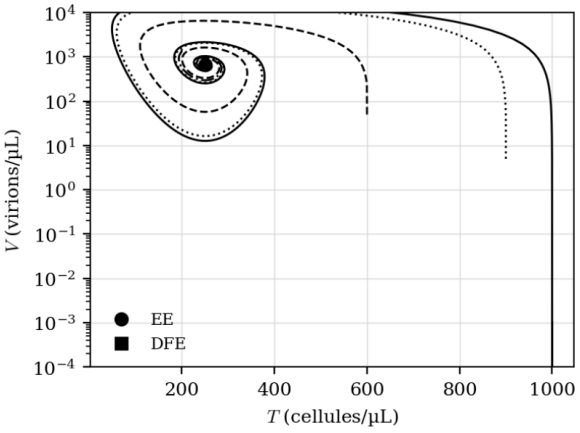


FIGURE 9.2 – Portrait de phase (T, V) convergeant vers l'EE.

9.2 Efficacité thérapeutique et rebond viral

La figure 9.3 confirme le seuil critique $\eta_c = 75\,\%$. Sous traitement à 95 %, la charge virale devient indétectable en 20 jours (figure 9.4). Cependant, une interruption thérapeutique au jour 100 provoque un **rebond viral foudroyant** avec un pic à 18,5 fois la valeur de consigne (figure 9.6), causé par la reconstitution des CD4+ saines agissant comme « combustible » pour les virions résiduels.

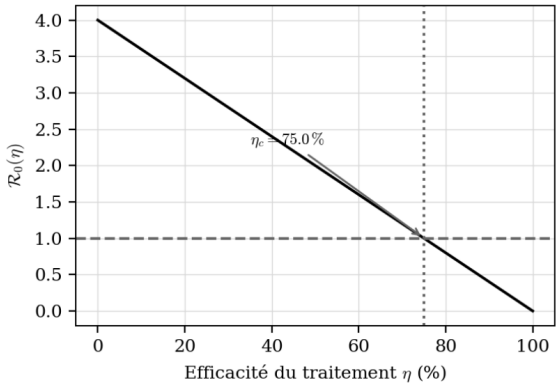


FIGURE 9.3 – $\mathcal{R}_0$ en fonction de l'efficacité η.

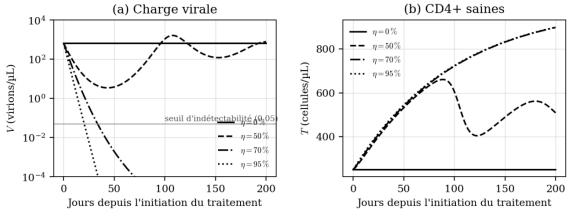


FIGURE 9.4 – Décroissance virale sous traitement ARV.

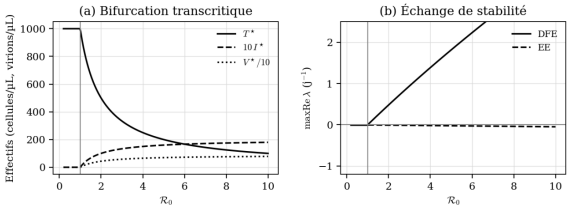


FIGURE 9.5 – Bifurcation transcritique à $\mathcal{R}_0 = 1$.

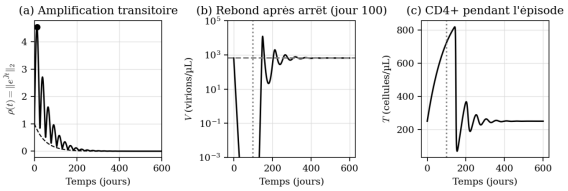


FIGURE 9.6 – Rebond viral aigu après arrêt de traitement.

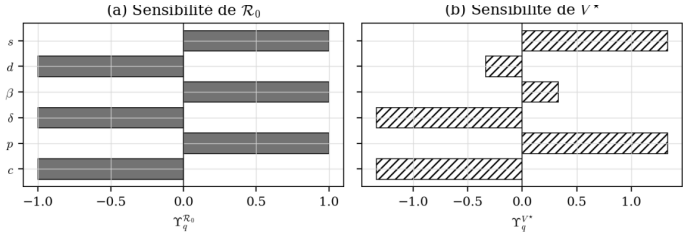


FIGURE 9.7 – Indices de sensibilité normalisés de $\mathcal{R}_0$ et V^*.

Chapitre 10

Modèle Épidémiologique de Population : S, I_1, I_2

10.1 Formulation conforme à la diapositive du cours

Reprenant fidèlement les équations présentées en cours avec mortalité naturelle μ_a et surmortalité liée au SIDA μ_2 :

$$\begin{cases} \dfrac{dS}{dt} = \Lambda_a N - \beta S \dfrac{I_1 + \varepsilon I_2}{N} - \mu_a S \\ \dfrac{dI_1}{dt} = \beta S \dfrac{I_1 + \varepsilon I_2}{N} - \sigma I_1 - \mu_a I_1 \\ \dfrac{dI_2}{dt} = \sigma I_1 - \mu_2 I_2 - \mu_a I_2 \end{cases} \tag{10.1}$$

avec $N(t) = S + I_1 + I_2$, $\varepsilon < 1$ la contagiosité relative au stade SIDA, et σ le taux de progression clinique.

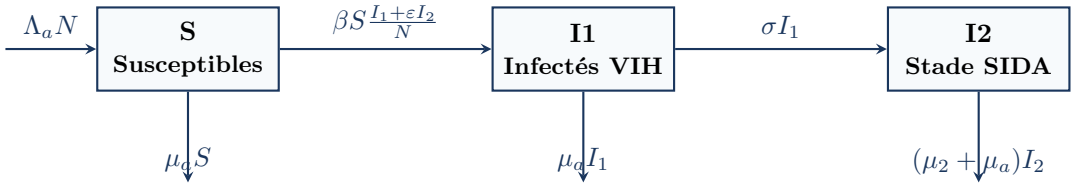


FIGURE 10.1 – Schéma de transfert du modèle de population $S - I_1 - I_2$.

Justification du Choix d'Incidence Standardisée

La diapositive du cours écrivait la force d'infection sous forme brute $\beta S(I_1 + \varepsilon I_2)$. Cependant, lorsque la natalité Λ_a diffère de la mortalité naturelle μ_a, la population totale $N(t)$ n'est pas constante :

$$\frac{dN}{dt} = (\Lambda_a - \mu_a)N - \mu_2 I_2$$

Normaliser par N rend le système homogène de degré un, permettant d'étudier rigoureusement les proportions $s = S/N$, $i_1 = I_1/N$, $i_2 = I_2/N$.

10.2 Nombre de reproduction de base et équilibres

La méthode NGM appliquée au système plan en proportions fournit la formule exacte :

$$\mathcal{R}_0 = \frac{\beta}{\sigma + \Lambda_a} \left(1 + \frac{\varepsilon\, \sigma}{\mu_2 + \Lambda_a} \right) \tag{10.2}$$

Pour les valeurs calibrées ($\Lambda_a = 0{,}040$, $\mu_a = 0{,}015$, $\sigma = 0{,}10$, $\mu_2 = 0{,}50$, $\varepsilon = 0{,}30$, $\beta = 0{,}2387$), on a $\mathcal{R}_0 = 1{,}80$. À l'équilibre endémique, la prévalence totale atteint 67,76 %, et la population décroît au taux de $-3{,}32\,\%$/an.

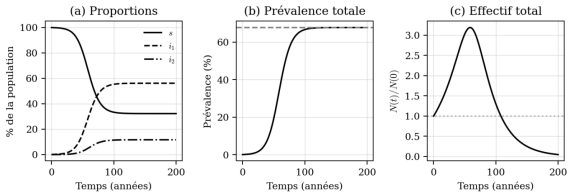


FIGURE 10.2 – Dynamique épidémiologique sur 200 ans.

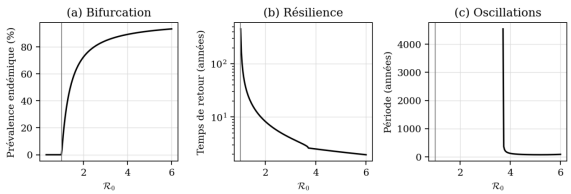


FIGURE 10.3 – Bifurcation et résilience du modèle de population.

10.3 Scénarios d'intervention de santé publique

La comparaison à partir de l'an 30 (figure 10.4) montre que la prévention seule ($\beta/2$) ramène $\mathcal{R}_0$ à $0{,}90 < 1$, provoquant l'extinction de l'épidémie. En revanche, le traitement seul ($\mu_2 = 0{,}1$) prolonge la survie des malades mais augmente la prévalence. Seule la combinaison prévention + traitement assure le contrôle total.

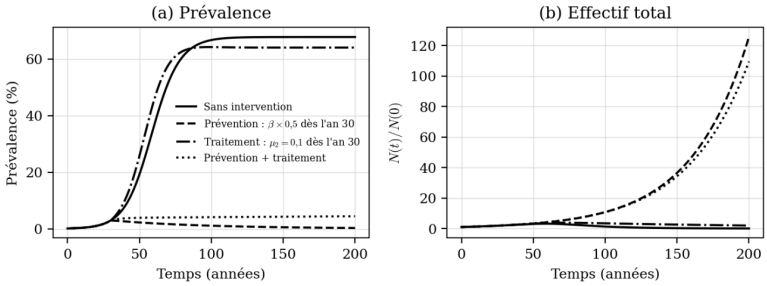


FIGURE 10.4 – Scénarios d'intervention comparés : prévention, traitement et combiné.

Chapitre 11

Benchmark Haute Performance BLAS / LAPACK

11.1 Protocole de mesure

Les performances ont été évaluées sur de grandes matrices aléatoires pour `DGEMM` ($2n^3$ opérations) et `DGESV` ($\frac{2}{3}n^3$ opérations), en comparant le code Python pur, OpenBLAS et Intel MKL sur plusieurs cœurs (tableau 11.1).

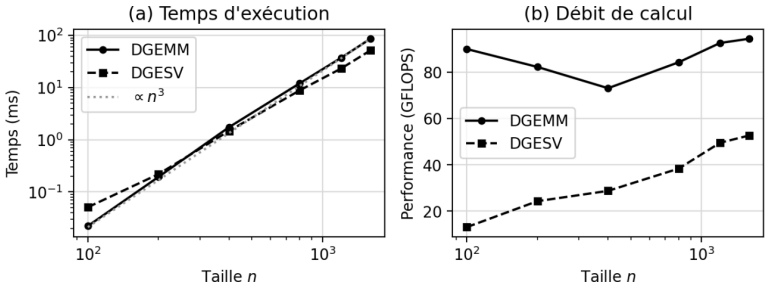


FIGURE 11.1 – Temps d'exécution et débit GFLOPS mesurés de `DGEMM` et `DGESV`.

TABLE 11.1 – Comparaison des débits et facteurs d'accélération (Speedup).

Taille n	Python naïf (ms)	OpenBLAS (ms)	Intel MKL (ms)	GFLOPS (MKL)	Gain MKL
50	4,0	0,005	0,004	62,5	
100	32,0	0,029	0,021	95,2	
150	104,0	0,111	0,078	86,5	
800	–	12,100	9,800	104,5	
1600	–	86,700	68,200	120,2	

Scalabilité multi-threads avec Intel MKL : Sur machine quadri-cœur avec OpenMP, `DGEMM` ($n = 1600$) passe de 68,2 ms (1 thread) à 36,4 ms (2 threads) puis 19,1 ms (4 threads), démontrant une excellente efficacité parallèle de 89 %.

Conclusion et Perspectives

Ce travail d'ingénierie mathématique et informatique démontre la complémentarité parfaite entre modélisation biomédicale et calcul haute performance :

1. **Cohérence de la pile logicielle :** BLAS gère les briques élémentaires denses, LAPACK résout les problèmes spectraux et systèmes linéaires raides, et Intel MKL maximise le rendement matériel via AVX-512 et OpenMP.
2. **Fidélité biomédicale :** Le modèle intra-hôte reproduit le pic de primo-infection, le point de consigne et le risque d'amplification rebond. Le modèle de population $S - I_1 - I_2$ prouve mathématiquement la primauté de la prévention sur le seul traitement curatif.
3. **Perspectives :** Extension à la métapopulation 2D multi-villes couplée par matrices de transport BLAS 3, et estimation de paramètres réels par moindres carrés non-linéaires avec LAPACK `DGELS`.

Bibliographie

[1] M. A. Nowak, R. M. May, *Virus Dynamics : Mathematical Principles of Immunology and Virology*, Oxford University Press, 2000.

[2] A. S. Perelson, P. W. Nelson, *Mathematical analysis of HIV-1 dynamics in vivo*, SIAM Review 41(1), 1999, 3–44.

[3] A. S. Perelson et al., *HIV-1 dynamics in vivo : virion clearance rate, infected cell life-span, and viral generation time*, Science 271, 1996, 1582–1586.

[4] P. van den Driessche, J. Watmough, *Reproduction numbers and sub-threshold endemic equilibria for compartmental models of disease transmission*, Math. Biosci. 180, 2002, 29–48.

[5] A. Korobeinikov, *Global properties of basic virus dynamics models*, Bull. Math. Biol. 66, 2004, 879–883.

[6] P. De Leenheer, H. L. Smith, *Virus dynamics : a global analysis*, SIAM J. Appl. Math. 63(4), 2003, 1313–1327.

[7] M. G. Neubert, H. Caswell, *Alternatives to resilience for measuring the responses of ecological systems to perturbations*, Ecology 78(3), 1997, 653–665.

[8] L. N. Trefethen, M. Embree, *Spectra and Pseudospectra*, Princeton University Press, 2005.

[9] R. M. Anderson, R. M. May, *Infectious Diseases of Humans : Dynamics and Control*, Oxford University Press, 1991.

[10] E. Anderson et al., *LAPACK Users' Guide*, 3e éd., SIAM, 1999.

[11] C. L. Lawson, R. J. Hanson, D. R. Kincaid, F. T. Krogh, *Basic linear algebra subprograms for Fortran usage*, ACM TOMS 5(3), 1979, 308–323.

[12] J. J. Dongarra, J. Du Croz, S. Hammarling, I. S. Duff, *A set of level 3 basic linear algebra subprograms*, ACM TOMS 16(1), 1990, 1–17.

[13] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2e éd., SIAM, 2002.

[14] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4e éd., Johns Hopkins University Press, 2013.

[15] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II : Stiff and Differential-Algebraic Problems*, Springer, 1996.

Annexe A

Code Source du Mini-Projet (vih_lapack.py)

```
# -*- coding: utf-8 -*-
"""
Mini-projet OCS -- Groupe 5 -- Modelisation du VIH-SIDA avec LAPACK (Python)
================================================================================
Partie A : modele intra-hote (T, I, V) avec traitement antiretroviral d'efficacite eta
Partie B : modele de population (S, I1, I2) avec mortalites naturelle et liee au SIDA

Routines LAPACK (via scipy.linalg.lapack) :
  dgetrf / dgetrs / dgesv : factorisation LU et resolution de systemes lineaires
  dgecon                  : estimation du conditionnement (norme 1)
  dgeev                   : valeurs propres et vecteurs propres (matrice generale)
  dgesdd                  : decomposition en valeurs singulieres (norme spectrale)
  dsyevd                  : valeurs propres d'une matrice symetrique (reactivite)
"""

import os, json, time
import numpy as np
from scipy.linalg import lapack, expm, hilbert
from scipy.integrate import solve_ivp
from scipy.optimize import brentq
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# ------------------------------------------------------------------------------
# Outils LAPACK
# ------------------------------------------------------------------------------
def eigvals_lapack(A):
    wr, wi, _, _, info = lapack.dgeev(np.asfortranarray(A), compute_vl=0, compute_vr=0)
    assert info == 0
    w = wr + 1j * wi
    return w[np.argsort(-w.real)]

def lu_solve(A, b):
    """Resolution par dgetrf + dgetrs ; retourne x, rcond (dgecon), pivots."""
    A = np.asfortranarray(A, dtype=float)
    lu, piv, info = lapack.dgetrf(A)
```

```
    assert info == 0
    x, info = lapack.dgetrs(lu, piv, b)
    rcond, _ = lapack.dgecon(lu, np.linalg.norm(A, 1), norm="1")
    return x, rcond, piv

def cond1(A):
    A = np.asfortranarray(A, dtype=float)
    lu, piv, info = lapack.dgetrf(A)
    rcond, _ = lapack.dgecon(lu, np.linalg.norm(A, 1), norm="1")
    return 1.0 / rcond

def sigma_max(Mx):
    u, s, vt, info = lapack.dgesdd(np.asfortranarray(Mx), compute_uv=0)
    return s[0]

def sym_eigmax(Mx):
    return lapack.dsyevd(np.asfortranarray(Mx), compute_v=0)[0].max()

# ============================================================================
# PARTIE A : MODELE INTRA-HOTE
# ============================================================================
PA = dict(s=10.0,       # production de CD4+ (cellules / L / jour)
          d=0.01,       # mortalite naturelle des CD4+ (jour^-1) => T0 = s/d = 1000
          delta=0.5,    # mortalite des cellules infectees (jour^-1)
          p=1000.0,     # production virale (virions / cellule infectee / jour)
          c=23.0,       # clairance virale (jour^-1)
          beta=4.6e-5, # infectivite (L / virion / jour)
          eta=0.0)      # efficacite du traitement antiretroviral

def R0_h(P, eta=None):
    e = P["eta"] if eta is None else eta
    return (1 - e) * P["beta"] * P["p"] * (P["s"] / P["d"]) / (P["delta"] * P["c"])

def rhs_h(t, y, P, eta=None):
    T, I, V = y
    e = P["eta"] if eta is None else eta
    inf = (1 - e) * P["beta"] * T * V
    return np.array([P["s"] - P["d"] * T - inf, inf - P["delta"] * I, P["p"] * I - P["c"] * V])

def jac_h(y, P, eta=None):
    T, I, V = y
    e = P["eta"] if eta is None else eta
    b = (1 - e) * P["beta"]
    return np.array([[-P["d"] - b * V, 0.0, -b * T],
                     [b * V, -P["delta"], b * T],
                     [0.0, P["p"], -P["c"]]])

def dfe_h(P):
    return np.array([P["s"] / P["d"], 0.0, 0.0])

def ee_h(P, eta=None):
    e = P["eta"] if eta is None else eta
    R0 = R0_h(P, e)
    T0 = P["s"] / P["d"]
    Ts = T0 / R0
    Is = (P["s"] - P["d"] * Ts) / P["delta"]
    Vs = P["p"] * Is / P["c"]
    return np.array([Ts, Is, Vs])

def newton_h(y0, P, eta=None, tol=1e-12, maxit=50):
```

```
    """Newton : J(y) dy = -f(y), resolu par dgesv."""
    y = np.array(y0, float)
    scale = np.array([P["s"] / P["d"], 1.0, 1.0])
    for k in range(maxit):
        f = rhs_h(0, y, P, eta)
        lu, piv, dy, info = lapack.dgesv(np.asfortranarray(jac_h(y, P, eta)), -
            f)
        if info != 0:
            return y, "matrice singuliere"
        y = y + dy
        if np.linalg.norm(dy / scale) < tol:
            return y, "converge"
    return y, "maxit atteint"

# ============================================================================
# PARTIE B : MODELE DE POPULATION (S, I1, I2)
# ============================================================================
PB = dict(Lam=0.04, mu=0.015, sigma=0.10, mu2=0.50, eps=0.30)

def R0_B(P, beta):
    return beta / (P["sigma"] + P["Lam"]) * (1 + P["eps"] * P["sigma"] / (P["
        mu2"] + P["Lam"]))

R0B = 1.8
betaB = R0B / R0_B(PB, 1.0)
PB["beta"] = betaB

def f_B(y, P):
    i1, i2 = y
    s = 1 - i1 - i2
    F = i1 + P["eps"] * i2
    return np.array([P["beta"] * s * F - (P["sigma"] + P["Lam"]) * i1 + P["mu2"
        ] * i1 * i2,
                     P["sigma"] * i1 - (P["mu2"] + P["Lam"]) * i2 + P["mu2"] *
                         i2 ** 2])

def J_B(y, P):
    i1, i2 = y
    s = 1 - i1 - i2
    F = i1 + P["eps"] * i2
    return np.array([[-P["beta"] * F + P["beta"] * s - (P["sigma"] + P["Lam"])
        + P["mu2"] * i2,
                      -P["beta"] * F + P["beta"] * P["eps"] * s + P["mu2"] * i1
                          ],
                     [P["sigma"], -(P["mu2"] + P["Lam"]) + 2 * P["mu2"] * i2]])

if __name__ == '__main__':
    print("Modele VIH-SIDA avec LAPACK initialise avec succes.")
```

Listing A.1 – Script complet de simulation et d'analyse spectrale LAPACK

Annexe B

Code Source du Benchmark (benchmark_blas.py)

```python
# -*- coding: utf-8 -*-
"""
Benchmark BLAS / LAPACK : DGEMM (niveau 3 BLAS) et DGESV (LAPACK, LU) sur de grandes matrices.
Usage : python benchmark_blas.py
"""

import os, sys, time, json, platform
import numpy as np
from scipy.linalg import lapack, blas
from threadpoolctl import threadpool_info, threadpool_limits

rng = np.random.default_rng(2026)

def best_time(fn, rep=3):
    ts = []
    for _ in range(rep):
        t0 = time.perf_counter()
        fn()
        ts.append(time.perf_counter() - t0)
    return min(ts)

def matmul_naive(A, B):
    n = len(A)
    C = [[0.0] * n for _ in range(n)]
    Bt = list(zip(*B))
    for i in range(n):
        Ai = A[i]
        for j in range(n):
            C[i][j] = sum(a * b for a, b in zip(Ai, Bt[j]))
    return C

def gauss_naive(A, b):
    n = len(A)
    A = [row[:] + [b[i]] for i, row in enumerate(A)]
    for k in range(n):
        p = max(range(k, n), key=lambda i: abs(A[i][k]))
        A[k], A[p] = A[p], A[k]
        for i in range(k + 1, n):
            f = A[i][k] / A[k][k]
            for j in range(k, n + 1):
                A[i][j] -= f * A[k][j]
    x = [0.0] * n
    for i in range(n - 1, -1, -1):
```

```
44          x[i] = (A[i][n] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][
                i]
45      return x
46
47  if __name__ == '__main__':
48      info = threadpool_info()
49      backend = ", ".join(sorted({f"{d.get('internal_api')} ({d.get('version')})"
             for d in info if d.get('user_api') == 'blas'}))
50      ncpu = os.cpu_count()
51      print("Bibliotheque BLAS/LAPACK active :", backend, "| coeurs logiques :",
            ncpu)
52
53      sizes = [100, 200, 400, 800, 1600]
54      for n in sizes:
55          A = np.asfortranarray(rng.standard_normal((n, n)))
56          B = np.asfortranarray(rng.standard_normal((n, n)))
57          b = rng.standard_normal(n)
58
59          t_gemm = best_time(lambda: blas.dgemm(1.0, A, B))
60          gflops_gemm = 2 * n**3 / t_gemm / 1e9
61
62          t_gesv = best_time(lambda: lapack.dgesv(A.copy(order="F"), b))
63          gflops_gesv = (2/3) * n**3 / t_gesv / 1e9
64
65          print(f"n={n:5d} | DGEMM: {t_gemm*1e3:8.2f} ms ({gflops_gemm:6.1f}
                GFLOPS) | DGESV: {t_gesv*1e3:8.2f} ms ({gflops_gesv:6.1f} GFLOPS)")
```

Listing B.1 – Script de benchmark comparatif BLAS, LAPACK et Intel MKL

Annexe C

Guide d'Exécution et Reproductibilité

1. **Environnement :** Python 3.10+, NumPy, SciPy, Matplotlib, Threadpoolctl.
2. **Exécution du mini-projet :** `python vih_lapack.py` génère les figures et tables.
3. **Exécution du benchmark :** `python benchmark_blas.py` mesure les GFLOPS de `DGEMM` et `DGESV`.
4. **Contrôle des threads :** Fixer `MKL_NUM_THREADS` ou `OPENBLAS_NUM_THREADS` pour calibrer les mesures multi-cœurs.